PAULA ARIAS. ISSI 1. COMANDOS MARIADB

1. CREACION DE UNA TABLA (CREATE TABLE)

Si te piden eliminar antes una tabla existente: **DROP TABLE IF EXISTS** NombreTabla;

```
CREATE TABLE   NombreTabla (
        CLAVE PRIMARIA(PK) -> id INT PRIMARY KEY AUTO_INCREMENT,
        AtributoString  VARCHAR (255) UNIQUE NOT NULL,
        AtributoEntero INT NOT NULL,
        AtributoFloat  DECIMAL(10, 2) NOT NULL,
        AtributoFecha DATE NOT NULL,
        AtributoTexto TEXT,
        AtributoBooleano BOOLEAN NOT NULL DEFAULT FALSE,
        AtributoEnum VARCHAR (64) CHECK (UPPER(AtributoEnum) IN ('VALOR1', 'VALOR2'))
        Ó Otra opción:  AtributoEnum ENUM ('VALOR1', 'VALOR2')
        FOREIGN KEY (FK)-> FOREIGN KEY (AtributoId) REFERENCES NombreTabla (id)
                ON DELETE CASCADE/RESTRICT/SET NULL
                ON UPDATE CASCADE/RESTRICT/SET NULL
        Reglas de validación->
        Caso 1: Validación simple
        CHECK (condición) Ej.:
        contraseña VARCHAR(255) NOT NULL CHECK (CHAR_LENGTH(contraseña) >= 8),
        Si necesitamos a ver un CHECK a la vez a dos columnas (ej. dos fechas), entonces usamos:
        CONSTRAINT fecha1menorafecha2 CHECK (fecha1<fecha2)
        Caso 2: Validación compleja (necesitas datos de otras tablas y mostrar mensajes)
        Ej.: (mas detalle en el punto 7)
        DELIMITER //
        CREATE TRIGGER cliente_edad_minima BEFORE INSERT ON Clientes
        FOR EACH ROW
        BEGIN
          IF TIMESTAMPDIFF(YEAR, NEW.fechaNacimiento, CURDATE()) <= 14 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'El cliente debe tener más de 14 años.';
          END IF;
        END //
        DELIMITER ;
```

2. INSERCIÓN DE DATOS (INSERT INTO)

INSERT INTO NombreTabla (Atributo1, Atributo2, Atributo3)

VALUES (valorAtributo1, ValorAtributo2, ValorAtributo3),

(valorAtributo1, ValorAtributo2, ValorAtributo3),

….

('cliente1@example.com', 120.00, 'yyyy-mm-dd'); *(por ejemplo)*

3. ACUALIZACIÓN DE DATOS (UPDATE)

UPDATE NombreTabla

SET Atributo1 = valor1,

Atributo2 = valor2,

…

WHERE condición1 AND condición2…; (MUY IMPORTANTE, AQUÍ SE PONE EL REGISTRO A CAMBIAR, SI NO PONES NADA SE CAMBIAN TODOS LOS REGISTROS)

Ej:

UPDATE Usuarios

SET email = 'nuevo@correo.com'

WHERE email = 'correo@ejemplo.com';

4. BORRADO DE DATOS (DELETE FROM)

DELETE FROM NombreTabla

WHERE condición AND condición2…; (MUY IMPORTANTE, AQUÍ SE PONE EL REGISTRO A BORRAR, SI NO PONES NADA SE BORRAN TODOS LOS REGISTROS DE LA TABLA)

EJ:

DELETE FROM Usuarios

WHERE email = 'correo@ejemplo.com';

Borrado con efectos en cascada: Son acciones que se ejecutan cuando intentas borrar (o actualizar) un registro que está relacionado con otros mediante una clave foránea (FOREIGN KEY).

ON DELETE CASCADE: Si borras un registro en la tabla padre, automáticamente se borran todos los registros relacionados en la tabla hija.

ON DELETE RESTRICT (por defecto): No te deja borrar un registro de la tabla padre si existen registros relacionados en la tabla hija.

ON DELETE SET NULL: Si borras un registro de la tabla padre, los registros de la tabla hija no se borran, pero su clave foránea se pone a NULL.

5. CONSULTA DE DATOS (SELECT)

SQL sigue una estructura **de consulta**, no de ejecución.

La idea es:

- **SELECT** → qué quieres
- **FROM** → de dónde
- **JOIN** → con qué otras tablas (tabulado después de FROM)
- **WHERE** → qué condiciones
- **GROUP BY** → cómo agrupar
- **ORDER BY** → cómo ordenar

Es como hablar:

"Quiero el nombre de los usuarios **que vienen de** la tabla Usuarios **unidos con** Pedidos **donde** el total sea mayor que 100."

Ejemplos de Consultas básicas de selección:

1.Lista todos los usuarios en la tabla `Usuarios`, mostrando solo sus nombres y correos electrónicos.

```
SELECT nombre, email FROM Usuarios;
```

Si ponemos SELECT * FROM Usuarios mostramos todos los atributos de Usuarios

Si ponemos SELECT DISTINCT nombre FROM Usuarios mostramos el atributo nombre sin duplicados de Usuarios

2.Muestra los empleados con su salario y el nombre correspondiente en la tabla `Usuarios`.

```
SELECT Usuarios.nombre, Empleados.salario FROM Empleados
JOIN Usuarios ON Empleados.usuarioId = Usuarios.id;
```

- **JOIN** → combina filas de dos tablas
- **Empleados.usuarioId** → clave foránea (FK) en la tabla Empleados
- **Usuarios.id** → clave primaria (PK) en la tabla Usuarios
- **ON** → condición que indica cómo deben emparejarse las filas

En otras palabras:

"Por cada empleado, busca el usuario cuyo id sea igual al usuarioId del empleado."

Siempre escribes JOIN seguido del *nombre de la tabla que quieres unir a la primera*. En este caso la primera tabla, en la que ya estamos, es Empleados (FROM Empleados).

3.Obtén los productos de la tabla `Productos` que tienen un precio mayor o igual a 20.00.

```
SELECT nombre, precio FROM Productos WHERE precio >= 20.00;
```

WHERE puede estar formada por:

- **Una combinación de AND, OR y NOT**
- **Operador EXISTS**
- **Operador IN (ej: WHERE salario IN (1000,2500,3000);**
- **Operadores ALL, ANY o SOME**
- **Operadores BETWEEN, IS NULL, LIKE (ej: WHERE salario BETWEEN 2000 AND 3000;)**

(WHERE nombre LIKE '_o%' OR jefe IS NULL;)

4. Lista los clientes con su dirección de envío, código postal y fecha de nacimiento.

```
SELECT Usuarios.nombre, Clientes.direccionEnvio, Clientes.codigoPostal, Clientes.fechaNacimiento
FROM Clientes
  JOIN Usuarios ON Clientes.usuarioId = Usuarios.id;
```

Ejemplos de Consultas con condiciones específicas y funciones agregadas

Encuentra el salario promedio de los empleados.

```
SELECT AVG(salario) AS salario_promedio FROM Empleados;
```

Obtén el número total de productos agrupados por tipo de producto.

```
SELECT TiposProducto.nombre AS tipo, COUNT(Productos.id) AS total_productos
FROM Productos
  JOIN TiposProducto ON Productos.tipoProductoId = TiposProducto.id
GROUP BY TiposProducto.nombre;
```

Lista los pedidos realizados por cada cliente con su nombre y la fecha de realización.

```
SELECT Usuarios.nombre AS cliente, Pedidos.fechaRealizacion
FROM Pedidos
  JOIN Clientes ON Pedidos.clienteId = Clientes.id
  JOIN Usuarios ON Clientes.usuarioId = Usuarios.id;
```

Cliente que ha pedido la menor variedad de productos.

```
SELECT u.nombre AS cliente, COUNT(DISTINCT lp.productoId) AS numero_productos_distintos
FROM Clientes c
 JOIN Pedidos p ON c.id = p.clienteId
 JOIN LineasPedido lp ON p.id = lp.pedidoId
 JOIN Usuarios u ON c.usuarioId = u.id
GROUP BY c.id, u.nombre
ORDER BY numero_productos_distintos ASC
LIMIT 1;
```

El orden real de ejecución SQL es:

1. **FROM**
2. **JOIN**
3. **ON**
4. **WHERE**
5. **GROUP BY**
6. **HAVING**
7. **SELECT**
8. **ORDER BY**

OPERACIONES CON VARIOS SELECT (CONSULTAS) -> UNION, INTERSECT, EXISTS, NOT EXISTS

SELECT * FROM Empleados E WHERE E.salario < 1000 UNION / INTERSECT SELECT * FROM Empleados E WHERE E.salario > 2000;

SELECT * FROM Departamentos D WHERE NOT EXISTS/ EXISTS (SELECT * FROM Empleados E WHERE D.departamentoId=E.departamentoId);

HAVING sirve para filtrar resultados después de hacer un GROUP BY, es decir, filtrar grupos, no filas individuales (para filas individuales usamos WHERE)

WHERE NO PUEDE usar funciones agregadas como:

- **COUNT()**
- **SUM()**
- **AVG()**
- **MAX()**
- **MIN()**

Para usar funciones agregadas usamos HAVING, de la siguiente forma, por ejemplo:

```
SELECT clienteId, COUNT(*) AS total_pedidos
FROM Pedidos
GROUP BY clienteId
HAVING COUNT(*) > 5;
```

Primero se filtran las filas mediante WHERE, luego se agrupan (GROUP BY), y luego se filtran los grupos mediante HAVING

ALL Y ANY

Compara una consulta con otra subconsulta (consulta anidada):

SELECT * FROM empleados WHERE salario > ALL / ANY (SELECT salario FROM empleados WHERE departamento = 'Ventas');

Ejemplos de Consultas con JOIN y filtrado avanzado

Encuentra los pedidos realizados por clientes mayores de edad (18 años o más).

```
SELECT Usuarios.nombre AS cliente, Pedidos.fechaRealizacion
FROM Pedidos
  JOIN Clientes ON Pedidos.clienteId = Clientes.id
  JOIN Usuarios ON Clientes.usuarioId = Usuarios.id
WHERE TIMESTAMPDIFF(YEAR, Clientes.fechaNacimiento, Pedidos.fechaRealizacion) >= 18;
```

Encuentra los productos que no han sido pedidos por ningún cliente menor de 18 años.

```
SELECT pr.id AS producto_id, pr.nombre
FROM Productos pr
WHERE pr.id NOT IN (
 SELECT lp.productoId
 FROM LineasPedido lp
  JOIN Pedidos p ON lp.pedidoId = p.id
  JOIN Clientes c ON p.clienteId = c.id
 WHERE TIMESTAMPDIFF(YEAR, c.fechaNacimiento, CURDATE()) < 18
                                                          );
```

> **TIMESTAMPDIFF** → calcula diferencias entre fechas.
> **CURDATE** → devuelve la fecha de hoy.
> **YEAR, MONTH, DAY, HOUR** → extrae el año, mes, día u hora de una fecha.

Muestra los pedidos y las líneas de pedido asociadas, incluyendo el nombre del producto y las unidades solicitadas.

```
SELECT Pedidos.id AS pedido_id, Productos.nombre AS producto, LineasPedido.unidades
FROM LineasPedido
  JOIN Pedidos ON LineasPedido.pedidoId = Pedidos.id
  JOIN Productos ON LineasPedido.productoId = Productos.id;
```

Lista los productos no permitidos para menores (`puedeVenderseAMenores = FALSE`) y sus precios.

```
SELECT nombre, precio
FROM Productos
WHERE puedeVenderseAMenores = FALSE;
```

Lista los clientes cuya mayoría de productos pedidos (en cantidad total de unidades) son aquellos no permitidos para menores de 18 años.

```
SELECT c.id AS cliente_id, u.nombre, u.email
FROM Clientes c
  JOIN Usuarios u ON c.usuarioId = u.id
  JOIN Pedidos p ON c.id = p.clienteId
  JOIN LineasPedido lp ON p.id = lp.pedidoId
  JOIN Productos pr ON lp.productoId = pr.id
GROUP BY c.id
HAVING
 (SELECT SUM(lp1.unidades)
  FROM LineasPedido lp1
   JOIN Productos pr1 ON lp1.productoId = pr1.id
   JOIN Pedidos p1 ON lp1.pedidoId = p1.id
  WHERE p1.clienteId = c.id AND pr1.puedeVenderseAMenores = FALSE) >
 (SELECT SUM(lp2.unidades)
  FROM LineasPedido lp2
   JOIN Productos pr2 ON lp2.productoId = pr2.id
   JOIN Pedidos p2 ON lp2.pedidoId = p2.id
  WHERE p2.clienteId = c.id AND pr2.puedeVenderseAMenores = TRUE);
```

Ejemplos de Consultas con subconsultas y cálculos

Encuentra el cliente con el mayor número de pedidos.

```
SELECT Usuarios.nombre AS cliente, COUNT(Pedidos.id) AS total_pedidos
FROM Pedidos
  JOIN Clientes ON Pedidos.clienteId = Clientes.id
  JOIN Usuarios ON Clientes.usuarioId = Usuarios.id
GROUP BY Usuarios.nombre
ORDER BY total_pedidos  DESC ----------→ (ORDER BY por defecto ordena en ASC)
LIMIT 1;
```

Muestra los pedidos con un total calculado de precio para cada pedido (sumando `precio * unidades` de `LineasPedido`).

```
SELECT Pedidos.id AS pedido_id, SUM(LineasPedido.precio * LineasPedido.unidades) AS total_precio
FROM Pedidos
  JOIN LineasPedido ON Pedidos.id = LineasPedido.pedidoId
GROUP BY Pedidos.id;
```

Muestra los pedidos cuyo importe sea el máximo. Si hay más de un pedido con el mismo importe máximo, devolverlos todos.

```
SELECT
        Pedidos.id AS pedido_id,
         SUM(LineasPedido.precio * LineasPedido.unidades) AS total_precio,
        (SELECT MAX(t.total)
          FROM (SELECT SUM(lp.precio * lp.unidades) AS total
                  FROM Pedidos p
                    JOIN LineasPedido lp ON p.id = lp.pedidoId
                  GROUP BY p.id) AS t) AS importe_maximo
FROM Pedidos
  JOIN LineasPedido ON Pedidos.id = LineasPedido.pedidoId
GROUP BY Pedidos.id
HAVING total_precio = importe_maximo;
```

Lista los productos que no se han vendido.

```
SELECT Productos.nombre
FROM Productos
  LEFT JOIN LineasPedido ON Productos.id = LineasPedido.productoId
WHERE LineasPedido.productoId IS NULL;
```

> **INNER JOIN** → solo filas con coincidencia.
>
> **LEFT JOIN** → todas las filas de la izquierda, coincidan o no.
>
> **RIGHT JOIN** → todas las filas de la derecha.
>
> **FULL JOIN** → todas las filas de ambas tablas.

Ganancias mensuales obtenidas.

```
SELECT MONTH(Pedidos.fechaRealizacion) AS Mes, SUM(LineasPedido.precio * LineasPedido.unidades)
AS total_precio
FROM Pedidos
  JOIN LineasPedido ON Pedidos.id = LineasPedido.pedidoId
GROUP BY MONTH(Pedidos.fechaRealizacion);
```

Empleado que ha gestionado más dinero en ventas.

```
SELECT usuarios.nombre AS Empleado, SUM(lineaspedido.precio * lineaspedido.unidades) AS dinero
FROM empleados
    JOIN usuarios ON empleados.usuarioId=usuarios.id
    JOIN pedidos ON pedidos.empleadoId=empleados.id
    JOIN lineaspedido ON lineaspedido.pedidoId=pedidos.id
GROUP BY empleados.id
ORDER BY dinero DESC
LIMIT 1;
```

Empleado que ha gestionado un total de más de 1000.00€ en ventas.

```
SELECT usuarios.nombre, SUM(lineaspedido.precio * lineaspedido.unidades) AS dinero
FROM empleados
    JOIN usuarios ON empleados.usuarioId=usuarios.id
    JOIN pedidos ON pedidos.empleadoId=empleados.id
    JOIN lineaspedido ON lineaspedido.pedidoId=pedidos.id
GROUP BY empleados.id
HAVING dinero > 1000.00;
```

5 pedidos con mayor importe de entre aquellos que tienen un importe menor al importe medio de todos los pedidos.

```
SELECT p.id AS pedidoId, SUM(lp.unidades * lp.precio) AS importeTotal
FROM Pedidos p
  JOIN LineasPedido lp ON p.id = lp.pedidoId
GROUP BY p.id
HAVING importeTotal < (
 SELECT AVG(total_importe)
 FROM (
    SELECT SUM(lp2.unidades * lp2.precio) AS total_importe
    FROM Pedidos p2
      JOIN LineasPedido lp2 ON p2.id = lp2.pedidoId
    GROUP BY p2.id
 ) AS importes
)
ORDER BY importeTotal DESC
LIMIT 5;
```

VISTAS para facilitar la creación de consultas sobre importes de pedidos y empleados encargados.

```
CREATE OR REPLACE VIEW v_importes_pedidos AS
SELECT pedidos.id AS pedidoId, empleados.id AS empleadoId, SUM(lineaspedido.unidades *
lineaspedido.precio) AS total_importe
FROM pedidos
  JOIN lineaspedido ON pedidos.id = lineaspedido.pedidoId
  JOIN empleados ON empleados.id=pedidos.empleadoId
GROUP BY pedidos.id;
```

CREATE OR REPLACE VIEW v_importes_pedidos AS

Crea una vista llamada **v_importes_pedidos**. Una vista es como una **consulta guardada**.

Clientes que han pedido en más de 3 meses distintos del último año uno de los 3 productos más vendidos de los últimos 5 años.

```
SELECT u.nombre, c.fechaNacimiento, u.email
FROM Clientes c
  JOIN Usuarios u ON c.usuarioId = u.id
  JOIN Pedidos p ON c.id = p.clienteId
  JOIN LineasPedido lp ON p.id = lp.pedidoId
  JOIN (
     SELECT productoId
     FROM LineasPedido
       JOIN Pedidos ON LineasPedido.pedidoId = Pedidos.id
     WHERE Pedidos.fechaRealizacion >= DATE_SUB(CURDATE(), INTERVAL 5 YEAR)
     GROUP BY productoId
     ORDER BY SUM(unidades) DESC
     LIMIT 3
   ) AS topProductos ON lp.productoId = topProductos.productoId
WHERE p.fechaRealizacion >= DATE_FORMAT(CURDATE(), '%Y-01-01')
GROUP BY c.id
HAVING COUNT(DISTINCT DATE_FORMAT(p.fechaRealizacion, '%Y-%m')) >= 3;
```

6. PROCEDIMIENTOS

Para modificar datos en las tablas. No se pueden usar en un SELECT, sino mediante CALL.

BLOQUE 1: CABECERA Y PARAMETROS

```
DELIMITER //
CREATE OR REPLACE PROCEDURE NombreProcedimiento (
    IN ParametroEntrada INT, VARCHAR(255)        INSTRUCCIONES SEPARADAS POR ,
)
BEGIN
```

BLOQUE 2: DECLARACION DE VARIABLES

```
    DECLARE variable INT, DATE      INSTRUCCIONES SEPARADAS POR ;
```

BLOQUE 3: CONSULTAS Y MANEJO DE ERRORES (pueden ir aquí las consultas que SOLO sirven para validar o preparar datos)

EJEMPLO DE CONSULTA:

```
SELECT p.clienteId INTO v_clienteId FROM Pedidos p WHERE p.id = NEW.pedidoId;
```

EJEMPLO DE MANEJO DE ERRORES (SENCILLO):

```
IF v_clienteId IS NULL THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El pedido no está asociado a un cliente válido.';
END IF;
```

EJEMPLO DE MANEJO DE ERRORES (DESHACEMOS TODO SI ALGO FALLA) SIEMPRE DELANTE DEL START TRANSACTION (BLOQUE 4):

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN ROLLBACK;
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error al registrar cliente premium';
END;
```

BLOQUE 4: EMPEZAMOS LA TRANSACCION

Cuando ejecutas:

```
START TRANSACTION;
```

Sucede lo siguiente:

1. Se desactiva el autocommit temporalmente (MySQL/MariaDB por defecto hace COMMIT después de cada sentencia).

2. Todas las operaciones posteriores (INSERT, UPDATE, DELETE…) quedan pendientes, sin guardarse definitivamente.

3. La transacción queda abierta hasta que hagas:

   COMMIT → guardar cambios o bien ROLLBACK → deshacer cambios

END //

DELIMITER ;

ESTRUCTURA DE PROCEDURE

```
DELIMITER //
CREATE OR REPLACE PROCEDURE NombreProcedimiento(
        IN ParametroEntrada INT, VARCHAR(255)
        ……
)
BEGIN
   DECLARE variable INT, DATE
    …….
  -- HANDLER DE ERRORES (CAPTURA ERRORES YA OCURRIDOS) (*)
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
     ROLLBACK;
     SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error en el procedimiento';
  END;
   START TRANSACTION;
   SELECT… INTO …. FROM …. WHERE..; (CONSULTAS)
   SET v_usuarioId = LAST_INSERT_ID(); (para unir el ID de una tabla con otro)
    IF …. THEN… ROLLBACK … SIGNAL SQLSTATE '45000' …. END IF; (VALIDACIONES)
    INSERT INTO VALUES/UPDATE SET WHERE/DELETE FROM WHERE ….. (ACCIONES)
    COMMIT;
END //
DELIMITER ;
```

(*) INCLUIRLO CUANDO USAMOS TRANSACCIONES O PROCEDIMIENTOS DE INSERT/UPDATE/DELETE

CUANDO EL PROCEDIMIENTO ES DE SOLO CONSULTA (SELECT) NO HACE FALTA

MUY IMPORTANTE: PARA EJECUTAR EL PROCEDIMIENTO HAY QUE USAR CALL:

CALL NombreProcedimiento(incluir todos los valores (no el nombre del parámetro) de los parámetros que hemos creado con IN, separados por comas);

7. FUNCIONES

Procedimiento que devuelve un valor. Usado para realizar cálculos que no modifican datos en tablas.

Se pueden usar en un SELECT (y WHERE).

ESTRUCTURA DE FUNCTION

```
DELIMITER //
CREATE FUNCTION
NombreFunción() RETURNS DOUBLE
BEGIN

DECLARE variable INT, DATE # EN ESTE CASO EL RETURN VA AL FINAL
RETURN (
SELECT AVG(comision)
FROM empleados
);
END //
DELIMITER ;
```

Las funciones las podemos usar luego dentro de los procedimientos y triggers. Se prueban con SELECT

8. TRIGGERS

Código asociado a eventos (como INSERT, UPDATE, DELETE en tablas). Se disparan automáticamente antes o después del evento al que se asocian. Usados para validación compleja de datos, reglas de negocio, mantenimiento de integridad de datos. Pueden llamar a funciones en su cuerpo, pero no procedimientos.

```
DELIMITER //
CREATE OR REPLACE TRIGGER tEmpleadoPropioJefe
BEFORE UPDATE ON Empleados FOR EACH ROW
BEGIN

DECLARE variable INT, DATE
IF (NEW.empleadoId = NEW.jefe) THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Un empleado no puede ser su propio jefe';
END IF;
END //
DELIMITER ;
```

Importante: Si el disparador es al actualizar, ponemos BEFORE UPDATE ON, si es para insertar datos BEFORE INSERT ON y si es para borrar BEFORE DELETE ON. Y usamos BEFORE el disparador debe saltar antes del evento o AFTER si debe saltar después del evento.

Acceso a valores: NEW.columna (nuevo valor, disponible en INSERT y UPDATE) y OLD.columna (valor anterior, disponible en UPDATE y DELETE)

9. VISTAS

Simplifican el acceso a datos complejos y consultas repetitivas. Actúan como tablas virtuales (pueden usarse en SELECT, JOIN, etc.). No almacenan datos propios, son una representación de una consulta sobre otras tablas.

```
CREATE VIEW ProductosMasPopulares AS
SELECT
p.id AS id_producto,
p.nombre AS nombre_producto,
SUM(v.cantidad) AS total_vendido
FROM Productos p
  JOIN Ventas v ON p.id = v.producto_id
GROUP BY p.id, p.nombre
ORDER BY total_vendido DESC;
```

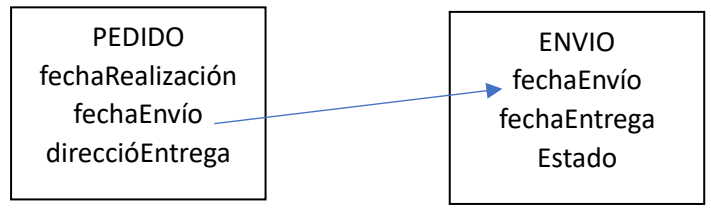
10. CUMPLIMIENTO DE LA REGLA 3FN

CUANDO NOS PIDAN INCLUIR UNA NUEVA TABLA, Y ESTA NUEVA TABLA DEBE TENER ALGUN CAMPO QUE YA ESTÁ EN OTRA TABLA, PARA CUMPLIR LA REGLA 3FN HAY QUE HACER LO SIGUIENTE:

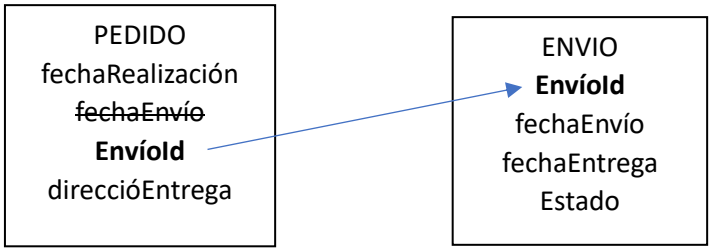
1. **CAMBIAR EL CAMPO QUE QUEREMOS PASAR DE UNA TABLA A LA OTRA POR SU campoId, EN LA TABLA EN LA QUE ESTABA INICIALMENTE**
2. **AÑADIR UNA FK PARA ENLAZAR EL campoId con el de la nueva tabla.**

EJEMPLO:

Tabla Actual ----------→ Añadimos Tabla Envío



No cumple la regla 3FN



Cumple la regla 3FN

11. COMANDOS

Funciones de FECHAS

Función	Descripción	Ejemplo
CURDATE	Fecha actual (sin hora)	`CURDATE()`
NOW	Fecha y hora actuales	`NOW()`
DATE_SUB	Resta tiempo a una fecha	`DATE_SUB(CURDATE(), INTERVAL 5 YEAR)`
DATE_ADD	Suma tiempo a una fecha	`DATE_ADD(NOW(), INTERVAL 2 DAY)`
DATE_FORMAT	Formatea una fecha	`DATE_FORMAT(fecha, '%Y-%m')`
YEAR	Extrae el año	`YEAR(fecha)`
MONTH	Extrae el mes	`MONTH(fecha)`
DAY	Extrae el día	`DAY(fecha)`
TIMESTAMPDIFF	Diferencia entre fechas	`TIMESTAMPDIFF(YEAR, fNac, CURDATE())`

Funciones de AGREGACIÓN

Función	Descripción	Ejemplo
SUM	Suma valores	`SUM(precio)`
COUNT	Cuenta filas	`COUNT(*)`
COUNT DISTINCT	Cuenta valores únicos	`COUNT(DISTINCT ciudad)`
AVG	Media	`AVG(salario)`
MAX	Máximo	`MAX(total)`
MIN	Mínimo	`MIN(total)`

Filtros y condiciones

Instrucción	Descripción	Ejemplo
WHERE	Filtra filas	`WHERE precio > 10`
HAVING	Filtra después del GROUP BY	`HAVING COUNT(*) > 3`
LIKE	Búsqueda por patrón	`nombre LIKE 'A%'`
IN	Lista de valores	`id IN (1,2,3)`
BETWEEN	Rango	`fecha BETWEEN '2026-01-01' AND '2026-12-31'`
DISTINCT	Elimina duplicados	`SELECT DISTINCT ciudad`

JOINS

Join	Descripción	Ejemplo
INNER JOIN	Solo coincidencias	`JOIN Usuarios u ON u.id=c.usuarioId`
LEFT JOIN	Todos los de la izquierda	`LEFT JOIN Pedidos p ON p.clienteId=c.id`
RIGHT JOIN	Todos los de la derecha	`RIGHT JOIN ...`
FULL JOIN	Todos de ambas tablas	*(no nativo en MySQL)*

Agrupación y orden

Instrucción	Descripción	Ejemplo
GROUP BY	Agrupa filas	`GROUP BY clienteId`
ORDER BY	Ordena resultados	`ORDER BY total DESC`
LIMIT	Limita filas	`LIMIT 3`

Funciones de cadenas y conversión

Función	Descripción	Ejemplo
CONCAT	Une cadenas o columnas	CONCAT(nombre, ' ', apellidos)
COALESCE	Devuelve el primer valor no nulo	COALESCE(apellidos, 'Sin datos')
CAST	Convierte tipos de datos	CAST(precio AS VARCHAR)

Funciones principales

Subconjunto	Comando	Función principal	Ejemplo
DDL (Definición de datos)	`CREATE`	Crear objetos (tablas, vistas, índices, procedimientos)	`CREATE TABLE Estudiantes (id INT, nombre VARCHAR(50));`
	`ALTER`	Modificar estructura de objetos	`ALTER TABLE Estudiantes ADD edad INT;`
	`DROP`	Eliminar objetos de la BD	`DROP TABLE Estudiantes;`
	`TRUNCATE`	Vaciar una tabla sin borrar su definición	`TRUNCATE TABLE Estudiantes;`
DML (Manipulación de datos)	`INSERT`	Insertar registros	`INSERT INTO Estudiantes VALUES (1, 'Ana', 20);`
	`UPDATE`	Modificar registros existentes	`UPDATE Estudiantes SET edad = 21 WHERE id = 1;`
	`DELETE`	Eliminar registros	`DELETE FROM Estudiantes WHERE id = 1;`
DQL (Consulta de datos)	`SELECT`	Consultar datos	`SELECT nombre, edad FROM Estudiantes;`
	`WHERE`	Filtrar registros	`SELECT * FROM Estudiantes WHERE edad > 18;`

	`ORDER BY`	Ordenar resultados	`SELECT * FROM Estudiantes ORDER BY edad DESC;`
	`GROUP BY`	Agrupar registros	`SELECT departamentoId, AVG(nota) FROM Estudiantes GROUP BY departamentoId;`
	`HAVING`	Filtrar grupos	`SELECT departamentoId, COUNT(*) FROM Estudiantes GROUP BY departamentoId HAVING COUNT(*) > 5;`
	`JOIN`	Combinar tablas	`SELECT * FROM Estudiantes JOIN Cursos ON Estudiantes.idCurso = Cursos.idCurso;`
	`IN`	Filtrar por lista de valores	`SELECT * FROM Estudiantes WHERE ciudad IN ('Sevilla','Cádiz');`
	`LIKE`	Buscar por patrón	`SELECT * FROM Estudiantes WHERE nombre LIKE 'A%';`
	`BETWEEN`	Filtrar por rango	`SELECT * FROM Estudiantes WHERE edad BETWEEN 20 AND 25;`
	`EXISTS`	Verificar existencia en subconsulta	`SELECT * FROM Departamentos D WHERE EXISTS (SELECT * FROM Empleados E WHERE D.id = E.idDepto);`

	`ANY`	Comparar con al menos un valor de una subconsulta	`SELECT * FROM Estudiantes WHERE nota > ANY (SELECT nota FROM Estudiantes WHERE cursoId=2);`
TCL (Control de transacciones)	`COMMIT`	Confirmar cambios	`COMMIT;`
	`ROLLBACK`	Deshacer cambios	`ROLLBACK;`
	`SAVEPOINT`	Crear punto de retorno	`SAVEPOINT punto1;`
	`SET TRANSACTION`	Definir propiedades de la transacción (ej. aislamiento)	`SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;`
DCL (Control de acceso)	`GRANT`	Otorgar permisos	`GRANT SELECT ON Estudiantes TO usuario1;`
	`REVOKE`	Revocar permisos	`REVOKE SELECT ON Estudiantes FROM usuario1;`